Raport

Autorzy: Natalia Misiaszek, Tomasz Martyński

Dane: https://www.kaggle.com/datasets/ahmeduzaki/earthquake-alert-prediction-dataset?resource=download

Pomoce: **Géron, A.** (rok wydania). *Uczenie maszynowe z użyciem Scikit-Learn i TensorFlow* (wyd. II). [Gliwice: Helion SA].

1. Wczytanie danych i wstępna analiza

W ramach projektu analizowany jest problem **klasyfikacji danych**, którego celem jest przewidywanie **koloru alertu trzęsienia ziemi** na podstawie wybranych cech sejsmicznych.

Wykorzystany zbiór danych zawiera informacje opisujące zdarzenia sejsmiczne oraz odpowiadające im alerty. Każda obserwacja opisana jest za pomocą pięciu zmiennych objaśniających:

- **magnitude** – wielkość trzęsienia ziemi w skali Richtera,
- **depth** – głębokość ogniska trzęsienia ziemi, rozumiana jako odległość od powierzchni Ziemi do punktu rozpoczęcia pękania,
- **cdi** – maksymalna zgłoszona intensywność trzęsienia ziemi w danym obszarze,
- **mmi** – maksymalna szacowana intensywność trzęsienia ziemi mierzona przyrządami,
- **sig** – złożony wskaźnik znaczenia trzęsienia ziemi, uwzględniający m.in. magnitudę, intensywność MMI, liczbę zgłoszeń od osób, które odczuły wstrząsy, oraz przewidywany wpływ zdarzenia. Wyższe wartości oznaczają bardziej istotne zdarzenia sejsmiczne.

Zmienną objaśnianą w projekcie jest **kolor alertu sejsmicznego**, przyjmujący cztery możliwe wartości: *green*, *yellow*, *orange* oraz *red*.

2. Cel projektu i metody badawcze

Celem projektu jest budowa i porównanie skuteczności wybranych modeli klasyfikacyjnych, które na podstawie danych treningowych będą w stanie **przewidywać kolor alertu trzęsienia ziemi** dla nowych obserwacji ze zbioru testowego.

W ramach analizy porównane zostaną następujące algorytmy:

- drzewo decyzyjne,
- las losowy (Random Forest),
- algorytm k najbliższych sąsiadów (k-NN),
- naiwny klasyfikator Bayesowski.

Ocena jakości modeli będzie opierać się przede wszystkim na **macierzy pomyłek**, a w wybranych eksperymentach — m.in. przy porównaniu wpływu normalizacji i standaryzacji Z-score — kluczową metryką będzie **dokładność (accuracy)**.

Zastosowanie dokładności jako miary jakości jest w tym przypadku uzasadnione, ponieważ zarówno zbiór treningowy, jak i testowy są **zbalansowane pod względem liczebności klas**. Dzięki temu metryka accuracy rzetelnie odzwierciedla ogólną skuteczność modeli i nie faworyzuje żadnej z klas decyzyjnych.

3. Przygotowanie danych

3.1. Sprawdzenie duplikatów

W kolejnym kroku zweryfikowano obecność zduplikowanych obserwacji, które mogłyby negatywnie wpłynąć na proces uczenia modeli. Po identyfikacji duplikatów utworzono oczyszczony zbiór danych (data_cleaned) zawierający wyłącznie unikalne rekordy. Zabieg ten pozwala zwiększyć wiarygodność późniejszych wyników klasyfikacji.

3.2. Analiza wartości odstających

Następnie przeprowadzono wstępną analizę wartości odstających (outliers) dla zmiennych numerycznych. W tym celu wykorzystano wykresy pudełkowe (boxploty), które umożliwiają wizualną identyfikację potencjalnych obserwacji odstających w poszczególnych cechach.

Analiza ta stanowi podstawę do dalszych decyzji dotyczących ewentualnego skalowania danych, normalizacji lub standaryzacji, które będą miały istotne znaczenie szczególnie dla algorytmów wrażliwych na skalę zmiennych, takich jak k-NN.

3.3 Podział na zbiór treningowy i testowy

Aby zapewnić losowy i proporcjonalny podział danych, wykorzystano funkcję sample, dzieląc zbiór danych na **zbiór treningowy (80%)** i **zbiór testowy (20%)**. Analiza proporcji klas w obu zbiorach wykazała, że są one bardzo podobne, a zatem możemy kontynuować trenowanie modeli, mając pewność, że **rozmieszczenie klas w zbiorze treningowym i testowym jest zrównoważone**.

4. Budowa i ocena modeli klasyfikacyjnych

4.1 Drzewo decyzyjne C5.0

4.1.1 Budowa i ocena modelu

- Ogólna dokładność modelu wynosi około **86,11%**, co świadczy o dobrej jakości klasyfikacji.
- Najważniejszą klasą z punktu widzenia bezpieczeństwa są **czerwone alerty**, odpowiadające sytuacjom największego zagrożenia. Model poprawnie rozpoznał **56 z 63 przypadków** (czułość ≈ 0,89), a 7 obserwacji zostało błędnie sklasyfikowanych.
- Klasy **pomarańczowa i żółta** były klasyfikowane stabilnie, zarówno pod względem czułości, jak i precyzji.
- Największe trudności model miał z klasą **zieloną**, dla której poprawnie sklasyfikowano tylko **48 z 64 przypadków**(czułość ≈ 0,75), co wskazuje na tendencję do **nadmiernej eskalacji alertów**.

4.1.2 Adaboost

Aby poprawić wyniki klasyfikacji, zastosowano metodę **AdaBoost** (boosting drzew decyzyjnych).

Wnioski:

- Ogólna dokładność modelu wynosi teraz około **86,9%**, a zatem lepiej niż w poprzednim przypadku.
- Boosting zmniejsza liczbę **najkosztowniejszych błędów** – żaden czerwony alert nie został sklasyfikowany jako zielony.
- 3 z 4 źle sklasyfikowanych czerwonych alertów zostały zaliczone do klasy **pomarańczowej**, co można uznać za dopuszczalne w kontekście bezpieczeństwa („mniejsze zło”).
- Tendencja do nadmiernej eskalacji pozostałych klas (zielona, żółta, pomarańczowa) nie zmieniła się istotnie, nadal jest wysoka.

4.1.3 Drzewo decyzyjne z macierzą kosztów

W algorytmie **C5.0** możliwe jest uwzględnienie **kosztów błędów klasyfikacji**, co pozwala na lepsze dostosowanie modelu do sytuacji, w której niektóre pomyłki są bardziej krytyczne.

Uwaga: Wartości w macierzy kosztów zostały ustalone **przykładowo, na potrzeby projektu**, aby zobrazować działanie tego mechanizmu. Nie oznacza to, że są one optymalne w sensie naukowym. W praktyce dobór hiperparametrów drzewa decyzyjnego, czy liczby drzew w boostingu został wykonany jednokrotnie i nie testowano wielu wariantów, tak jak to miało miejsce w przypadku algorytmu k-NN. W projekcie najważniejszy jest **sens podejścia**, a nie powtarzanie tych samych eksperymentów wielokrotnie. Co więcej drzewo decyzyjne z

boostingiem i hiperparametrem trials=10 i tak daje już lepsze wyniki, niż bez boostingu, chociaż możnaby rzec delikatnie lepsze.

Wnioski:

- Wyniki są podobne do klasycznego drzewa decyzyjnego bez boostingu.
- Spośród trzech wariantów (**drzewo standardowe, AdaBoost, drzewo z macierzą kosztów**) najlepszy efekt daje **boosting**, szczególnie w kontekście minimalizacji najbardziej kosztownych błędów (czerwony alert → zielony alert).
- Zastosowana macierz kosztów pełni funkcję **ilustracyjną**, pokazując mechanizm uwzględniania kosztów w klasyfikacji. Być może z realnymi kosztami, podanymi np przez sejsmologów, model miałby wyższą dokładność i więcej sensu.

4.2 Las losowy

Las losowy (Random Forest) jest metodą uczenia zespołowego, która buduje wiele drzew decyzyjnych i łączy ich predykcje w celu zwiększenia dokładności klasyfikacji. W przeciwieństwie do **AdaBoost**, gdzie klasyfikatory są trenowane sekwencyjnie, Random Forest działa **równolegle**, co poprawia stabilność modelu i zmniejsza ryzyko nadmiernego dopasowania.

Uwaga: W projekcie przyjęto konkretne parametry modelu (ntree=100, mtry=2) bez testowania wielu wariantów. Do oceny Random Forest, jak i tak naprawdę pozostałych modeli bardziej skuteczne byłoby stosowanie walidacji krzyżowej lub w ogólności odpowiednich funkcji pakietu **caret**, ale w naszym projekcie najważniejszy był **sens podejścia**, a nie optymalizacja hiperparametrów „na wszystkie możliwe sposoby”, jak to miało miejsce w k-NN.

Wnioski:

- Ogólna dokładność modelu wynosi teraz około **91,27%**, a zatem jak najlepiej do tej pory.
- Random Forest **dobrze klasyfikuje wszystkie klasy**, w szczególności **czerwone alerty**, które są kluczowe z punktu widzenia bezpieczeństwa.
- Najgorsze przypadki, w których realne zagrożenie mogłoby zostać błędnie sklasyfikowane jako zielony alert, **nie występują**, co stanowi istotną poprawę względem poprzednich modeli.
- Najwięcej błędów pojawia się w klasie **zielonej**, ale są to błędy „sąsiednie” (zielona ↔ żółta), nie stanowiące zagrożenia i nadal dają najlepszy wynik w porównaniu z wcześniejszymi modelami, czułość wzrosła do ok 0,7969;
- Tendencja do nadmiernej eskalacji poziomu alertów jest znacznie mniejsza niż w przypadku pojedynczych drzew decyzyjnych czy AdaBoost.

4.3 k-NN

Metoda k-NN jest algorytmem klasyfikacji opartym na odległościach między obserwacjami. Z tego względu koniecznym było **przeskalowanie danych** (normalizacja min-max do przedziału [0,1]) w celu zapewnienia, że wszystkie cechy mają podobny wpływ na obliczanie odległości, zwłaszcza posiadając wiedzę o występowaniu wartości odstających.

Uwaga: Normalizację przeprowadzono **po podziale na zbiór treningowy i testowy**, aby uniknąć wycieku danych – dane testowe nie mogły w żaden sposób wpływać na proces trenowania.

Wnioski wstępne:

- Dokładność modelu wynosi około **73,02%**.
- Model unika błędów fałszywie negatywnych dla **najgroźniejszych przypadków** („red"), co jest kluczowe z punktu widzenia bezpieczeństwa. Jeżeli już myli red z którąś klasą, to jest to klasa pomarańczowa, więc znowu możemy określić to jako "najmniejsze zło".
- Zaniżanie poziomu zagrożenia wystąpiło jedynie **13 razy**, zawsze o jedną klasę różnicy, co świadczy o zachowawczym charakterze modelu.

4.3.1 Optymalizacja parametru k

Aby dobrać wartość parametru **k**, sprawdzono dokładność modelu dla różnych wartości.

Wnioski z eksperymentu:

- Mniejsze wartości k prowadzą do wyższej dokładności, ale bardzo małe k (1,3,5) mogą powodować **przeuczenie**.
- Wybrano **k=7** jako kompromis między dokładnością a minimalizacją kosztownych błędów.
- Model dla k=7 utrzymuje stabilną skuteczność, przy czym błędy dotyczą głównie **sąsiednich klas** („orange" ↔ „yellow") – mniej kosztownych niż przeoczenie realnego zagrożenia. Dokładność modelu zwiększyła się do około **81,75%**.

4.3.2 Standaryzacja z-score

Dodatkowo sprawdzono wpływ **standaryzacji z-score** na dokładność modeli k-NN, w porównaniu do wcześniej stosowanej **normalizacji min-max**.

Wnioski z eksperymentu:

- Dokładność modeli k-NN ze standaryzacją jest generalnie **niższa niż przy normalizacji min-max**.

- Modele zarówno częściej popełniają **błędy fałszywie negatywne**, czyli zaniżają poziom zagrożenia, co jest niekorzystne w kontekście bezpieczeństwa, jak i mają większą tendencję do nadmiernej eskalacji poziomu problemu.

4.3.3 Wnioski końcowe dla k-NN

- Najlepszym modelem k-NN jest **k=7 z normalizowanymi danymi min-max**.
- Dobór parametru k przeprowadzono **ręcznie**, porównując dokładność dla różnych wartości k na zbiorze testowym.
- To podejście pozwala znaleźć **kompromis między skutecznością a ryzykiem fałszywie negatywnych klasyfikacji**, bez stosowania formalnej walidacji krzyżowej.
- Model skutecznie minimalizuje najgroźniejsze błędy („red"), a większość pomyłek dotyczy **klas sąsiadujących**, co jest mniej kosztowne.

4.4 Naiwny klasyfikator Bayesowski (NB)

Naiwny klasyfikator Bayesowski (NB) świetnie sprawdza się na danych **kategorycznych**. Można go stosować również na danych ilościowych, zakładając, że każda zmienna przyjmuje **rozklad normalny w danej klasie**. Jednak brak spełnienia tego założenia może prowadzić do **błędnych estymacji prawdopodobieństw** i niskiej jakości predykcji.

W praktyce, mimo naruszenia założenia o niezależności cech, NB często działa dobrze, ponieważ w klasyfikacji istotny jest **porządek prawdopodobieństw między klasami**, a nie ich dokładne wartości.

4.4.1 Analiza cech i rozkładów zmiennych

- **Niezależność cech:** Wizualizacja macierzy korelacji pokazuje, że większość cech nie wykazuje silnej korelacji, choć pojawiają się wyjątki (ciemniejsze bloki). Nie jest to jednak sytuacja krytyczna.
- **Rozkłady zmiennych ilościowych:** Dla każdej klasy sprawdzono gęstość zmiennych takich jak magnitude, depth, cdi, mmi, sig. Wizualizacja pokazuje, że przy braku dyskretyzacji założenie normalności często nie jest spełnione, co może pogorszyć wyniki modelu.

4.4.2 NB bez dyskretyzacji

Wyniki:

- Dokładność: ~ **63,49%** – najniższa spośród rozważanych modeli.
- Model przyjmuje rozkład normalny dla zmiennych ilościowych, co w tym przypadku prowadzi do niedokładnych predykcji.

4.4.3 NB z dyskretyzacją

Dane ilościowe zostały podzielone na **kwantyle** (funkcja quantilize) w celu dyskretyzacji cech:

Działanie funkcji quantilize:

1. Wyznacza granice kwantyli na podstawie danych treningowych (np. 4 przedziały = 5 kwantyli).
2. Tworzy etykiety dla każdego przedziału (np. Q1, Q2, …).
3. Zastosowuje ten sam podział zarówno dla zbioru treningowego, jak i testowego, aby **zapobiec wyciekowi danych**.
4. W naszym przypadku dla zbioru testowego I kolumny depth pojawiły się dwie wartości NA, to znaczy dwie próbki ze zbioru testowego nie zostały przyporządkowane do żadnego przedziału międzykwantylowego, co może mieć bezpośredni związek z występującymi w kolumnie depth outliersami.

Budowa i ocena modelu NB po dyskretyzacji:

- Dokładność: ~ **64.68%** – niewielka poprawa względem modelu bez binningu.
- Najgroźniejsze błędy są nadal obecne, choć część drobnych pomyłek została ograniczona.

4.4.4 Manewrowanie parametrem Laplace'a

Parametr **Laplace** służy do wygładzania zerowych prawdopodobieństw w NB.

Przetestowano modele z binningiem i z parametrem Laplace'a z zakresu {0,1,2,3,4,5,6,7,8}.

Wnioski:

- Wartość domyślna **Laplace = 0** jest wystarczająca.
- Dla wartości parametru Laplace w zakresie od 0 do 6 dokładność modelu pozostaje taka sama, jak w przypadku NB z binningiem. Zwiększanie parametru nie poprawia dokładności, a nadmierne wygładzanie może ją nieznacznie obniżać.

4.4.5 Podsumowanie

- Naiwny klasyfikator Bayesowski osiąga **najniższą dokładność spośród wszystkich modeli (~ 64%)**.
- Dyskretyzacja danych ilościowych poprawia wyniki minimalnie, ale nie rozwiązuje problemu błędnego założenia normalności.
- Parametr Laplace'a nie wymaga modyfikacji.

5. Zestawienie porównawcze zastosowanych modeli

Model	Wariant / Parametry	Skalowanie danych	Dokładność (≈)	Charakter błędów	Ocena ogólna
Drzewo decyzyjne (C5.0)	Parametry domyślne	brak	~86,11%	Sporadyczne zaniżanie alertów „red”; eskalacja „green”	Dobry punkt odniesienia
Drzewo decyzyjne + AdaBoost	trials = 10	brak	~86,9%	Brak klasyfikacji „red” jako „green”; tendencja do nadmiernego eskalowania poziomu alertów	Bezpieczniejsze niż pojedyncze drzewo
Drzewo decyzyjne + macierz kosztów	Koszty dobrane arbitralnie	brak	~85,71%	Podobne do drzewa bazowego	Brak istotnej poprawy
Random Forest	ntree = 100, mtry = 2	brak	najwyższa	Brak krytycznych błędów; pomyłki „sąsiednie”, najwyższa dokładność, najwięcej “zer”	**Najlepszy model**
k-NN	k = 31	normalizacja min–max	~73,02%	Model unika błędów fałszywie negatywnych dla **najgroźniejszych przypadków** („red”); zachowawczy charakter	Model bazowy
k-NN	k = 7	normalizacja min–max	~81,75%	Rzadkie błędy dla „red”, głównie klasy sąsiednie	Najlepszy wariant k-NN
k-NN	k = 7, 15	standaryzacja z-score	niższa	Częstsze zaniżanie alertów	Gorsze niż normalizacja

Model	Wariant / Parametry	Skalowanie danych	Dokładność (≈)	Charakter błędów	Ocena ogólna
Naive Bayes	bez binningu	brak	~63,49%	Częste błędy krytyczne, tendencja do nadmiernego eskalowania poziomu alertów	Słaby
Naive Bayes	binning (kwantyle)	brak	~64,68%	Brak istotnej poprawy	Słaby
Naive Bayes	binning + Laplace	brak	~64–65%	Nadmierne wygładzanie	Brak sensu regulacji

---

5.1 Wnioski

1. **Najlepszy model pod kątem ogólnej skuteczności i bezpieczeństwa predykcji**: **Random Forest** – wysoka dokładność, brak katastrofalnych błędów, błędy ograniczone do klas sąsiednich.
2. **Drzewo decyzyjne z AdaBoost** – dobra alternatywa, poprawia błędy krytyczne w stosunku do prostego drzewa C5.0.
3. **k-NN (k=7)** – zachowawcze podejście minimalizujące krytyczne błędy, ale z niższą dokładnością ogólną.
4. **Naiwny Bayes** – najniższa dokładność, użycie dyskretyzacji i regulacji Laplace'a nie przynosi znaczącej poprawy.

W kontekście zastosowania praktycznego, gdzie bezpieczeństwo predykcji jest kluczowe (czerwone alerty), **Random Forest** jest najlepszym rozwiązaniem, łączącym wysoką skuteczność klasyfikacji z minimalnym ryzykiem błędów krytycznych.